House Price Prediction

- Trong phần này, bạn sẽ học được cách sử dụng dữ liệu sẵn có để xây dựng một mô hinhf dự đoán (dự đoán giá nhà).

  > - Xây dựng/ Khởi tạo mô hình như thế nào?

- Đưa dữ liệu vào huấn luyện mô hình?
- Sử dụng mô hình đã huấn luyện để dự đoán?

- Dữ liệu sử dụng: clean_data.csv

Load dữ liệu và tiền xử lí trước khi huấn luyện

1. Import thư viện cần thiết

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)
```

2. Load dữ liệu cần thiết

In [4]:
```
path = "./clean_data.csv"
df = pd.read_csv(path)
df.shape
```

Out[4]:
```
(7120, 6)
```

In [5]:
```
df.head()
```

Out[5]:

	bath	balcony	price	total_sqft_float	bhk	price_per_sqft
0	3.0	2.0	150.0	1672.0	3	8971.291866
1	3.0	3.0	149.0	1750.0	3	8514.285714
2	3.0	2.0	150.0	1750.0	3	8571.428571
3	2.0	2.0	40.0	1250.0	2	3200.000000
4	2.0	2.0	83.0	1200.0	2	6916.666667

3. Phân chia dữ liệu train - test

In [6]:
```
# Xác định thông tin thuộc tính X và nhãn y
X = df.drop("price", axis=1)
y = df['price']
print('Shape of X = ', X.shape)
print('Shape of y = ', y.shape)
```

```
Shape of X =  (7120, 5)
Shape of y =  (7120,)
```

In [7]:
```
#Chia dữ liệu - train_test_split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s
print('Shape of X_train = ', X_train.shape)
print('Shape of y_train = ', y_train.shape)
print('Shape of X_test = ', X_test.shape)
print('Shape of y_test = ', y_test.shape)
```

```
Shape of X_train =  (5696, 5)
Shape of y_train =  (5696,)
Shape of X_test =  (1424, 5)
Shape of y_test =  (1424,)
```

4. Feature Scaling

> If feature scaling is not done, then a machine learning algorithm tends to weigh greater values, higher and consider smaller values as the lower values, regardless of the unit of the values.

In [8]:
```
#Chuẩn hóa giá trị của các feature trong 1 phạm vi nào đó.

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
sc.fit(X_train)
X_train= sc.transform(X_train)
X_test = sc.transform(X_test)
```

Machine Learning Model Training

Linear Regression

In [9]:
```
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Lasso
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
lr = LinearRegression()
lr_lasso = Lasso()
lr_ridge = Ridge()
```

In [10]:
```
def rmse(y_test, y_pred):
  return np.sqrt(mean_squared_error(y_test, y_pred))
```

In [11]:
```
lr.fit(X_train, y_train)
lr_score = lr.score(X_test, y_test) # with all num var 0.7842744111909903
lr_rmse = rmse(y_test, lr.predict(X_test))
lr_score, lr_rmse
```

Out[11]:
```
(0.7837532911322952, np.float64(65.91685277030534))
```

In [12]:
```
# Lasso
lr_lasso.fit(X_train, y_train)
lr_lasso_score=lr_lasso.score(X_test, y_test) # with balcony 0.5162364637824872
lr_lasso_rmse = rmse(y_test, lr_lasso.predict(X_test))
lr_lasso_score, lr_lasso_rmse
```

Out[12]:
```
(0.8015775415270464, np.float64(63.14182401879733))
```

In [13]:
```
# Ridge
lr_ridge.fit(X_train, y_train)
lr_ridge_score = lr_ridge.score(X_test, y_test) # with all num var 0.78427441119099
lr_ridge_rmse = rmse(y_test, lr_ridge.predict(X_test))
lr_ridge_score, lr_ridge_rmse
```

Out[13]:
```
(0.7839330766013356, np.float64(65.88944574857665))
```

Support Vector Machine

In [14]:
```
from sklearn.svm import SVR
svr = SVR()
svr.fit(X_train,y_train)
svr_score=svr.score(X_test,y_test)
svr_rmse = rmse(y_test, svr.predict(X_test))
svr_score, svr_rmse
```

Out[14]:
```
(0.24613512350275257, np.float64(123.07460910013376))
```

Random Forest

In [15]:
```
from sklearn.ensemble import RandomForestRegressor
rfr = RandomForestRegressor()
rfr.fit(X_train,y_train)
rfr_score=rfr.score(X_test,y_test)
rfr_rmse = rmse(y_test, rfr.predict(X_test))
rfr_score, rfr_rmse
```

Out[15]:
```
(0.8867809060429713, np.float64(47.69595488446046))
```

Test Model

In [16]:
```
def predict_house_price(model,bath,balcony,total_sqft_int,bhk,price_per_sqft,area_t

  x =np.zeros(len(X.columns))

  x[0]=bath
  x[1]=balcony
  x[2]=total_sqft_int
  x[3]=bhk
  x[4]=price_per_sqft

  if "availability"=="Ready To Move":
    x[8]=1

  if 'area_type'+area_type in X.columns:
    area_type_index = np.where(X.columns=="area_type"+area_type)[0][0]
    x[area_type_index] =1

  if 'location_'+location in X.columns:
    loc_index = np.where(X.columns=="location_"+location)[0][0]
    x[loc_index] =1

  x = sc.transform([x])[0]

  return model.predict([x])[0]
```

area_type	availability	location	bath	balcony	total_sqft_int	bhk	price_per_sqft
Plot Area	Ready to Move	Devarabeesana	3	2	1672	3	8971.291866

In [17]:
```
# Test Linear Regression
lr_test = predict_house_price(model=lr, bath=3,balcony=2,total_sqft_int=1672,bhk=3,
print("Test Linear Regression: ", lr_test)
# Test Lasso
lr_lasso_test = predict_house_price(model=lr_lasso, bath=3,balcony=2,total_sqft_int
print("Test Lasso: ", lr_lasso_test)
```

```
Test Linear Regression:  164.5115052528608
Test Lasso:  165.8477470710708
```

```
c:\Users\Admin\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWar
ning: X does not have valid feature names, but StandardScaler was fitted with featur
e names
  warnings.warn(
c:\Users\Admin\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWar
ning: X does not have valid feature names, but StandardScaler was fitted with featur
e names
  warnings.warn(
```

In [18]:
```
# Test SVM
svm_test = predict_house_price(model=svr, bath=3,balcony=2,total_sqft_int=1750,bhk=
print("Test SVM: ", svm_test)
```

```
Test SVM:  152.21389416918345
```

```
c:\Users\Admin\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWar
ning: X does not have valid feature names, but StandardScaler was fitted with featur
e names
  warnings.warn(
```

In [19]:
```
# Test Random Forest
test_random_forest = predict_house_price(model=rfr,bath=3,balcony=3,total_sqft_int=
print("Test Random Forest: ", test_random_forest)
```

```
Test Random Forest:  148.96
```

```
c:\Users\Admin\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWar
ning: X does not have valid feature names, but StandardScaler was fitted with featur
e names
  warnings.warn(
```

Save model & load model

In [20]:
```
import joblib
joblib.dump(rfr, 'bangalore_house_price_prediction_model.pkl')
```

Out[20]:
```
['bangalore_house_price_prediction_model.pkl']
```

In [21]:
```
bangalore_house_price_prediction_model = joblib.load("bangalore_house_price_predict
```

In [22]:
```
predict_house_price(bangalore_house_price_prediction_model,bath=3,balcony=3,total_s
```

```
c:\Users\Admin\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2749: UserWar
ning: X does not have valid feature names, but StandardScaler was fitted with featur
e names
  warnings.warn(
```

Out[22]:
```
np.float64(36.25)
```